

corridor

Build Your Own Bridge

A USDC bridge app on Circle CCTP V2, built the way bridge.setsuna.finance is: your users' own wallets, any CCTP chain with Sui included, your fee, and a choice of who pays the gas.

day2-corridor.vercel.app

[KEYS, DOLLARS AND BITCOIN · DAY 2 · SUIHUB LAGOS]

RURU (@MINTING_RURU)

[THE ONE IDEA]

A bridge is a burn, a proof and a mint.

CCTP does all three.
You build the product around it.

CCTP burns native USDC on one chain, Circle's attesters sign the message, and the other chain mints the same amount. No wrapped token, no pool, nothing for you to guard. Your product is everything around it: wallets, routes, fees, gas and a status screen that tells the truth. V1 contracts are paused on 1 December 2026, so this is V2.

[THE PRODUCT]

What a bridge app does

The six parts of bridge.setsuna.finance. Our app has the same six.

01 Connect

One wallet per ecosystem, side by side: MetaMask for EVM chains, Slush for Sui.

02 Route

Any two CCTP chains. Each has a domain, a USDC address and two contracts.

03 Quote

Amount, fees and what arrives, shown before anything is signed.

04 Speed

Fast costs a small fee. Standard is free. Sui is standard only, and fast anyway.

05 Gas

The user claims on the destination, Circle delivers for USDC, or you pay.

06 Track

Approve, burn, attest, mint. Saved at every step, found again by hash.

[CHECKED 8 TO 10 NOVEMBER]

CCTP V2 on Sui: what is live

From Circle's docs, then confirmed on chain: Sui testnet, Sui mainnet and Base Sepolia.

	What we found	How we know
Domains	Ethereum 0, Arbitrum 3, Solana 5, Base 6, Sui 8	Circle docs, Iris messages
V2 on Sui testnet	Live: TMM 0x267d…3c64, MT 0xe967…bbe	GraphQL types, our burns
V2 on Sui mainnet	Live: TMM 0xeb14…b830, MT 0x16bc…db87	Burn CtgJ…bvnb, mint Gy33…Hoq8
Sui as source	Standard only, attested in 5 s	Our timed burn C1qH…bB5T
Into Sui	Fast works, 1 bp fee	Live mint DHwH…yad
V1	Burn limits cut 31 Oct, paused 1 Dec 2026	Circle blog, 27 Aug
Fee API	No quotes for any route touching Sui	Iris sandbox, 10 Nov

[ONE TRANSFER]

Four steps on every route

Approve, burn, attest, mint. Only the code for each step changes from chain to chain.

01

Approve

EVM only: let the token messenger spend the USDC. Sui needs none: the coin goes into the PTB.

02

Burn

depositForBurn on EVM, a three-call PTB on Sui. Names the destination domain and recipient.

03

Attest

Ask Circle's Iris API with the source domain and burn hash until it is signed. A 404 means not indexed yet.

04

Mint

receiveMessage on EVM, a four-call PTB on Sui. Whoever submits it pays the gas.

05

Save

Write the record after every step, so a closed tab never loses a transfer.

[SRC/LIB/WALLETS.TSX]

One wallet per ecosystem

Abridged. EVM wallets announce themselves (EIP-6963). Sui wallets register with Wallet Standard.

TYPESCRIPT

```typescript
// EVM: every wallet announces itself (EIP-6963)
window.addEventListener('eip6963:announceProvider', (e) => {
  const { info, provider } = (e as CustomEvent).detail;
  evm.set(info.rdns, { name: info.name, icon: info.icon, provider });
});
window.dispatchEvent(new Event('eip6963:requestProvider'));

// Sui: wallets register with Wallet Standard (Slush, Suiet, ...)
const sui = getWallets().get()
  .filter((w) => w.chains.some((c) => c.startsWith('sui:')));

// Connect, one per ecosystem
const [address] = await provider.request({ method: 'eth_requestAccounts' });
const { accounts } = await slush.features['standard:connect'].connect();
connected = { evm: { address, provider }, sui: { address: accounts[0].address, wallet: slush } };
```

[SRC/LIB/KIT.TS]

EVM to EVM with Bridge Kit

Circle's SDK runs approve, burn, attestation and mint for you, with events for the tracker.

TYPESCRIPT

```ts
import { BridgeKit } from '@circle-fin/bridge-kit';
import { createViemAdapterFromProvider } from '@circle-fin/adapter-viem-v2';

const kit = new BridgeKit();
const adapter = await createViemAdapterFromProvider({ provider: metamask });
kit.on('*', (e) => track(e.method, e.values?.txHash)); // approve, burn, fetchAttestation, mint

const result = await kit.bridge({
  from: { adapter, chain: 'Base_Sepolia' },
  to: gasInUsdc
    ? { chain: 'Arbitrum_Sepolia', recipientAddress, useForwarder: true } // Circle mints
    : { adapter, chain: 'Arbitrum_Sepolia' },                             // the user claims
  amount: '1.00',
  config: { transferSpeed: 'FAST' },
});
if (result.state !== 'success') await kit.retry(result, { from: adapter, to: adapter });
```

[SRC/LIB/SUI.TS]

Sui routes: your own PTBs

Abridged. Out of Sui is three calls the user's Sui wallet signs. Into Sui is four.

TYPESCRIPT

```typescript
// Sui → anywhere. Finality 2000: Sui is standard only, and final in a second.
const coin = tx.add(coinWithBalance({ type: USDC, balance: amount }));
const [receipt, rest] = tx.moveCall({ target: `${TMM}::deposit_for_burn::deposit_for_burn`,
  typeArguments: [USDC], arguments: [coin, tx.pure.u32(destDomain), tx.pure.address(recipient32),
    tx.pure.address('0x0'), tx.pure.u256(maxFee), tx.pure.u32(2000), tx.pure.vector('u8', []),
    tx.object(TMM_STATE)] });
const [ticket] = tx.moveCall({ target: `${HANDLER}::handler::burn`,
  arguments: [tx.object(HANDLER_STATE), receipt, rest, tx.object('0x403'), tx.object(TREASURY)] });
tx.moveCall({ target: `${TMM}::deposit_for_burn::complete_burn`,
  typeArguments: [USDC, `${HANDLER}::handler::Auth`], arguments: [ticket, tmmState, mtState] });
await tx.prepareForSerialization({ client });          // resolve the coins, then the wallet signs

// anywhere → Sui, after Iris attests:
// receive_message → prepare_mint<USDC> → handler::mint → complete_mint<USDC, Auth>
```

[THE GAS]

Who pays on the other side

The mint costs gas on the destination. Three ways to pay it; the app offers what each route supports.

	Who submits the mint	User needs there	Works on
Claim it yourself	The user's wallet	Gas (ETH, SUI)	Every route
Pay gas in USDC	Circle Forwarding Service	Nothing	EVM routes only
Corridor pays	Our relayer, the destination caller	Nothing, 0 SUI to send	Sui routes, for our fee

Circle's fee API answered "Invalid source/destination domain id" for every Sui route on 10 Nov. For Sui, you run the relayer.

[THE BUSINESS]

Your fee: Bridge Kit or Move

Bridge Kit facts from Circle's docs on 6 November. Corridor facts from our contract and runs.

Bridge Kit fee (EVM routes)

- setCustomFeePolicy with a computeFee callback
- Taken on the source chain by the SDK
- Circle keeps 10 percent of your fee
- No Sui adapter

Move fee (Sui routes)

- corridor::send, in the same PTB as the burn
- Measured from the coin, kept in an onchain vault
- CCTP names your package as the sender
- Pays for the sponsored gas and the relayer

[A TRANSFER, START TO FINISH]

Half done is a state, not an error

What the tracker shows at each stage. Close the tab anywhere: the burn hash brings it back.

01

Burn sent

Save the hash before waiting. A retry settles it first: never two burns.

02

Attesting

Iris answers 404 or pending. Show pending, never failed.

03

Ready

Attested, not minted. Claim now, or wait for Circle or the relayer.

04

Minted

The nonce is used on the destination. Check before you pay gas.

05

Find it

Source domain and burn hash: Iris returns the message on any device.

[VERIFIED]

Proof: real transfers

Every route through the live app on 10 November 2026, plus someone else's transfer on Sui mainnet.

Route	Chain	Transaction
Bridge Kit burn with our fee	Base Sepolia	`0x1cd7eea61631906e...d83b3c5d3d52`
Base to Arbitrum, Circle delivers	Arbitrum Sepolia	`0x306a45f03eb6058f...e74cdcc1fcea`
Sui to Base, user claims	Base Sepolia	`0xcc7a695a1b479b64...88cb2979d5ba`
Base to Sui, user claims	Sui testnet	`CnYYh73wTjCZUyWnMFQFYVMRad3apzCwFGEaMM1DnrKy`
Sui to Base, Corridor pays (0 SUI)	Base Sepolia	`0xccbe62a560ef9237...cbf2b5d29ddc`
Base to Sui, Corridor pays	Sui testnet	`4Mrr1e2GmZ6YrgmR5gmtvAWmSsByv2v8Umr5hHqfZCU2`
Mainnet: someone's Sui mint	Sui mainnet	`Gy3393NZCGnzxt88vPUjK6bjmSpWkqVjujjXvkwYoHq8`

[GOTCHAS]

Six things that will bite you

All of them came up building Corridor and bridge.setsuna.finance.

01 V2 is not V1

On Sui the handler burns the coin. Bridge Kit only knows Sui's V1 contracts.

02 Sui is standard only

Fast is N/A from Sui, and the fee API has no Sui quotes.

03 Prepare, then sign

Resolve coinWithBalance with a client before the wallet sees the PTB.

04 404 is pending

Iris has not indexed the burn yet. Never show failed.

05 Save the burn first

Record the hash before the receipt, or a retry burns twice.

06 Check the recipient

Sui is 32 bytes, EVM 20. Circle mints to whatever you give it.

Take home: CCTP moves the dollars. Your app is the wallets, the route, the gas and the truth about a half-done transfer.